AURA-FACE: Developer Bible & Architecture Manual

1. Cover Page & Architecture Foundations

This exhaustive manual details the technical foundations, mathematical limits, and file-by-file logic of the AURA-FACE system.

Dependency Matrix

Python 3.10+, PyTorch (MobileNetV2), OpenCV (YuNet), FAISS (IndexFlatL2 wrapped in IndexIDMap2), MediaPipe (Face Mesh), CustomTkinter, and SQLite3.

2. Mathematical Derivations & Thresholds

Eye Aspect Ratio (EAR) Liveness

EAR prevents spoofing attacks using 468 3D landmarks.
Formula: EAR = (||p2-p6|| + ||p3-p5||) / (2 * ||p1-p4||).
Landmark Topologies (Right Eye): 362, 385, 387, 263, 373, 380.
Landmark Topologies (Left Eye): 33, 160, 158, 133, 153, 144.
Blink State Machine triggers closed state at < 0.20 and open state at > 0.25.

High-dimensional Vector Space & L2 Normalization

Vectors generated by the PyTorch embedder are 128-dimensional. These are strictly L2-normalized so they map onto a unit hypersphere. Because vectors are unit length, the Squared Euclidean Distance used by FAISS mathematically directly maps to Cosine Similarity: L2_Sq = 2 - 2*cos(theta).

Dual-Threshold Gating

Strict Duplicate Blocking: Distance 0.15 (High similarity required to block true duplicates but allow lookalikes).
Attendance Kiosk Match: Distance 0.55 (Standard tolerance accommodating lighting/expression variations).

3. Module-by-Module Code Breakdown

Module: face_engine.py

```
import os
import cv2
cv2.utils.logging.setLogLevel(cv2.utils.logging.LOG_LEVEL_ERROR)
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.models as models
from torchvision import transforms
from PIL import Image

class MobileNetV2Embedder(nn.Module):
def __init__(self):
super(MobileNetV2Embedder, self).__init__()
# Load base model without weights
base_model = models.mobilenet_v2(weights=None)

# Feature extractor
self.features = base_model.features

# Pooling layer
self.pool = nn.AdaptiveAvgPool2d((1, 1))

# Custom head
self.head = nn.Sequential(
nn.Dropout(p=0.2),
nn.Linear(base_model.last_channel, 256),
nn.ReLU(),
nn.Linear(256, 128)
)

def forward(self, x):
# Extract features
x = self.features(x)
# Apply adaptive pooling
x = self.pool(x)
# Flatten
x = torch.flatten(x, 1)
# Pass through head
x = self.head(x)
# Apply L2 normalization
x = F.normalize(x, p=2, dim=1)
return x


class FaceEngine:
def __init__(self,
detector_path='models/face_detection_yunet_2023mar.onnx',
embedder_path='models/mobilenetv2_feature_extractor.pth'):
# Initialize device strictly on CPU
self.device = torch.device("cpu")

# Load YuNet Detector
if not os.path.exists(detector_path):
print(f"[Warning] Detector model not found at {detector_path}. Detection will fail if called.")
self.detector = None
else:
self.detector = cv2.FaceDetectorYN.create(
detector_path,
"",
(640, 480),
score_threshold=0.8,
nms_threshold=0.3
)

# Load MobileNetV2 Embedder
self.embedder = MobileNetV2Embedder()
if os.path.exists(embedder_path):
self.embedder.load_state_dict(torch.load(embedder_path, map_location=self.device))
else:
print(f"[Warning] Embedder weights not found at {embedder_path}. Using random weights.")
```

```
self.embedder.to(self.device)
self.embedder.eval()

# Image Preprocessing Transform
self.transform = transforms.Compose([
transforms.Resize((160, 160)),
transforms.ToTensor(),
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

def detect_faces(self, frame_bgr):
if self.detector is None:
raise RuntimeError("Detector model was not loaded properly.")

# Dynamically set input size based on current frame shape
h, w = frame_bgr.shape[:2]
self.detector.setInputSize((w, h))

# Detect faces
_, faces = self.detector.detect(frame_bgr)
return faces

def extract_embedding(self, face_crop_bgr) -> np.ndarray:
# Convert BGR crop to RGB
face_crop_rgb = cv2.cvtColor(face_crop_bgr, cv2.COLOR_BGR2RGB)

# Create PIL Image
pil_img = Image.fromarray(face_crop_rgb)

# Apply transforms
tensor_img = self.transform(pil_img).unsqueeze(0).to(self.device)

# Run forward pass through the embedder
with torch.no_grad():
emb = self.embedder(tensor_img)

# Return a 128-dimensional float32 NumPy array
return emb.squeeze().cpu().numpy().astype(np.float32)


if __name__ == '__main__':
print("=== STEP 4: face_engine.py SELF-TEST ===")

try:
# 1. Initialize FaceEngine
engine = FaceEngine()

# 2. Feed a synthetic mock frame through detection
mock_frame = np.random.randint(0, 256, (480, 640, 3), dtype=np.uint8)
print("Feeding mock frame of shape:", mock_frame.shape)
```

```
if engine.detector is not None:
faces = engine.detect_faces(mock_frame)
print("Detected faces:", faces if faces is not None else "None")
else:
print("Skipped face detection test due to missing model.")

# 3. Run a dummy face crop through extract_embedding
mock_face_crop = np.random.randint(0, 256, (112, 112, 3), dtype=np.uint8)
embedding = engine.extract_embedding(mock_face_crop)

# 4. Confirm the output shape is (128,)
print(f"Embedding shape: {embedding.shape}")
print(f"Embedding dtype: {embedding.dtype}")

if embedding.shape == (128,):
print("[OK] Embedding extraction successful!")
else:
print("[ERROR] Embedding shape mismatch.")

except Exception as e:
print(f"[ERROR] Self-test failed: {e}")
print("=== SELF-TEST COMPLETE ===")
```

Module: liveness_detector.py (File not found)

Module: vector_index.py

```
import os
import faiss
import numpy as np

class VectorIndex:
def __init__(self, index_path='models/face_index.faiss', dim=128):
self.index_path = index_path
self.dim = dim
self._load_or_create_index()

def _load_or_create_index(self):
if os.path.exists(self.index_path):
try:
self.index = faiss.read_index(self.index_path)
print(f"[VectorIndex] Loaded existing index from {self.index_path} with {self.index.ntotal}
vectors.")
except Exception as e:
print(f"[VectorIndex] Failed to load index: {e}. Creating a new one.")
self._create_new_index()
else:
self._create_new_index()

def _create_new_index(self):
# Create a new IndexIDMap2 wrapping an IndexFlatL2
# IndexIDMap2 is required to store integer IDs alongside vectors natively in FAISS
flat_index = faiss.IndexFlatL2(self.dim)
self.index = faiss.IndexIDMap2(flat_index)
print(f"[VectorIndex] Initialized fresh FAISS IndexIDMap2(IndexFlatL2({self.dim})).")
self.save()

def add_vector(self, user_id: int, embedding: np.ndarray):
"""
Normalizes and adds a 128-dimensional embedding to the FAISS index.
"""
# Ensure embedding is 2D array of shape (1, 128) and float32
if embedding.ndim == 1:
embedding = np.expand_dims(embedding, axis=0)

embedding = np.ascontiguousarray(embedding, dtype=np.float32)

if embedding.shape[1] != self.dim:
raise ValueError(f"Embedding dimension mismatch. Expected {self.dim}, got {embedding.shape[1]}")

# Normalize the embedding using L2 norm
faiss.normalize_L2(embedding)

# FAISS expects IDs as int64
id_array = np.array([user_id], dtype=np.int64)

# Add to index
self.index.add_with_ids(embedding, id_array)
self.save()
return True

def remove_vector(self, user_id: int):
"""Removes the vector associated with the given user_id from the index."""
if self.index.ntotal == 0:
return False
selector = faiss.IDSelectorArray(np.array([user_id], dtype=np.int64))
self.index.remove_ids(selector)
self.save()
return True

def search(self, query_emb: np.ndarray, threshold=0.55):
"""
Searches the index for the closest vector to query_emb.
Returns (matched_user_id, distance) if successful, else (None, distance).
"""
if self.index.ntotal == 0:
return None, float('inf')

# Ensure query is 2D array and float32
```

```
if query_emb.ndim == 1:
query_emb = np.expand_dims(query_emb, axis=0)

query_emb = np.ascontiguousarray(query_emb, dtype=np.float32)

# Normalize the query embedding for cosine similarity equivalent using L2
faiss.normalize_L2(query_emb)

# Search for top 1 closest vector
distances, indices = self.index.search(query_emb, k=1)

matched_user_id = int(indices[0][0])
distance = float(distances[0][0])

if matched_user_id != -1 and distance <= threshold:
return matched_user_id, distance

return None, distance

def save(self):
"""Saves the current index to disk."""
os.makedirs(os.path.dirname(self.index_path), exist_ok=True)
faiss.write_index(self.index, self.index_path)

if __name__ == '__main__':
print("=== STEP 3: vector_index.py SELF-TEST ===")

# Use a dummy path for the self-test
test_index_path = 'models/test_face_index.faiss'
vi = VectorIndex(index_path=test_index_path, dim=128)

# 1. Create a mock embedding and add to index
mock_id = 999
np.random.seed(42)
mock_emb = np.random.randn(128).astype(np.float32)

print(f"1. Adding mock vector for User ID {mock_id}...")

vi.add_vector(mock_id, mock_emb)
print(f" [OK] Vector added. Total vectors in index: {vi.index.ntotal}")

# 2. Perform a search query with the exact same embedding
print(f"2. Querying the exact same vector...")
found_id, dist = vi.search(mock_emb.copy(), threshold=0.55)

if found_id == mock_id:
print(f" [OK] Match successful! Found ID {found_id} with distance {dist:.5f}")
else:
print(f" [FAIL] Search returned ID {found_id} with distance {dist:.5f}")

# 3. Query with random noise (should fail the threshold)
print(f"3. Querying with random noise...")
noise_emb = np.random.randn(128).astype(np.float32)
found_id_noise, dist_noise = vi.search(noise_emb, threshold=0.55)

if found_id_noise is None:
print(f" [OK] Noise successfully rejected. Distance {dist_noise:.5f} > threshold 0.55")
else:
print(f" [FAIL] Noise was incorrectly matched to ID {found_id_noise} with distance
{dist_noise:.5f}")

# Clean up test index
if os.path.exists(test_index_path):
os.remove(test_index_path)

print("=== SELF-TEST COMPLETE ===")
```

Module: db_manager.py

```
import sqlite3
import hashlib
from datetime import datetime

DB_PATH = "attendance.db"

def init_db():
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE IF NOT EXISTS admins (
username TEXT PRIMARY KEY,
password_hash TEXT NOT NULL
)
''')

cursor.execute('''
CREATE TABLE IF NOT EXISTS users (
id INTEGER PRIMARY KEY AUTOINCREMENT,
user_code TEXT UNIQUE NOT NULL,
name TEXT NOT NULL,
registered_date TEXT NOT NULL
)
''')

cursor.execute('''
CREATE TABLE IF NOT EXISTS attendance (
entry_id INTEGER PRIMARY KEY AUTOINCREMENT,
user_code TEXT,
name TEXT,
date TEXT,
time TEXT,
UNIQUE(user_code, date)
)
''')

cursor.execute("SELECT * FROM admins WHERE username = 'admin'")
if not cursor.fetchone():
hashed_pw = hashlib.sha256("admin123".encode()).hexdigest()
cursor.execute("INSERT INTO admins (username, password_hash) VALUES (?, ?)", ("admin", hashed_pw))

conn.commit()
conn.close()

def verify_admin(username, password) -> bool:
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
hashed_pw = hashlib.sha256(password.encode()).hexdigest()
cursor.execute("SELECT * FROM admins WHERE username = ? AND password_hash = ?", (username,
hashed_pw))
result = cursor.fetchone() is not None
conn.close()
return result

def add_user(user_code, name, reg_date=None) -> int:
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()

if not reg_date:
reg_date = datetime.now().strftime('%Y-%m-%d')

try:
cursor.execute("INSERT INTO users (user_code, name, registered_date) VALUES (?, ?, ?)",
(user_code, name, reg_date))
conn.commit()
numeric_id = cursor.lastrowid
return numeric_id
except sqlite3.IntegrityError:
return -1
finally:
conn.close()
```

```
def update_user(numeric_id, new_user_code, new_name, new_reg_date=None):
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
try:
cursor.execute("SELECT user_code FROM users WHERE id = ?", (numeric_id,))
row = cursor.fetchone()
if not row:
return False, "User not found."

old_user_code = row[0]

if new_reg_date:
cursor.execute("UPDATE users SET user_code = ?, name = ?, registered_date = ? WHERE id = ?",
(new_user_code, new_name, new_reg_date, numeric_id))
else:
cursor.execute("UPDATE users SET user_code = ?, name = ? WHERE id = ?",
(new_user_code, new_name, numeric_id))

cursor.execute("UPDATE attendance SET user_code = ?, name = ? WHERE user_code = ?",
(new_user_code, new_name, old_user_code))

conn.commit()
return True, "User updated successfully."
except sqlite3.IntegrityError:
return False, "User code already exists."
except Exception as e:
return False, str(e)
finally:
conn.close()

def delete_user(numeric_id: int):
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
try:
cursor.execute("SELECT user_code FROM users WHERE id = ?", (numeric_id,))
row = cursor.fetchone()
if not row:
return False, "User not found."

user_code = row[0]

cursor.execute("DELETE FROM users WHERE id = ?", (numeric_id,))
cursor.execute("DELETE FROM attendance WHERE user_code = ?", (user_code,))

conn.commit()
return True, "User deleted successfully."
except Exception as e:
return False, str(e)
finally:
conn.close()

def get_all_users() -> list:
conn = sqlite3.connect(DB_PATH)
```

```
cursor = conn.cursor()
cursor.execute("SELECT id, user_code, name, registered_date FROM users ORDER BY id ASC")
results = cursor.fetchall()
conn.close()
return results

def get_user_by_id(numeric_id) -> tuple:
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
cursor.execute("SELECT user_code, name FROM users WHERE id = ?", (numeric_id,))
result = cursor.fetchone()
conn.close()
return result

def mark_attendance(user_code, name):
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
now = datetime.now()
date_str = now.strftime('%Y-%m-%d')
time_str = now.strftime('%H:%M:%S')

try:
cursor.execute("INSERT INTO attendance (user_code, name, date, time) VALUES (?, ?, ?, ?)",
(user_code, name, date_str, time_str))
conn.commit()
return True, time_str
except sqlite3.IntegrityError:
# Already punched in today
# Fetch the existing time
cursor.execute("SELECT time FROM attendance WHERE user_code = ? AND date = ?", (user_code,
date_str))
existing_time = cursor.fetchone()[0]
return False, existing_time
finally:
conn.close()

def get_today_attendance() -> list:
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
date_str = datetime.now().strftime('%Y-%m-%d')
cursor.execute("SELECT user_code, name, date, time FROM attendance WHERE date = ? ORDER BY entry_id
DESC", (date_str,))
results = cursor.fetchall()
conn.close()
return results

def get_daily_attendance_stats() -> dict:
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
date_str = datetime.now().strftime('%Y-%m-%d')

cursor.execute("SELECT COUNT(id) FROM users")
total = cursor.fetchone()[0] or 0

cursor.execute("SELECT COUNT(entry_id) FROM attendance WHERE date = ?", (date_str,))
present = cursor.fetchone()[0] or 0

absent = max(0, total - present)
percentage = round((present / total * 100), 1) if total > 0 else 0.0

conn.close()
return {'total': total, 'present': present, 'absent': absent, 'percentage': percentage}

def get_user_attendance_summary() -> list:
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()

cursor.execute("SELECT COUNT(DISTINCT date) FROM attendance")
total_working_days = cursor.fetchone()[0] or 0

cursor.execute("SELECT id, user_code, name FROM users ORDER BY id ASC")
users = cursor.fetchall()
```

```
summary = []
for u in users:
uid, user_code, name = u
cursor.execute("SELECT COUNT(DISTINCT date) FROM attendance WHERE user_code = ?", (user_code,))
present_days = cursor.fetchone()[0] or 0
absent_days = max(0, total_working_days - present_days)
percentage = f"{round((present_days / total_working_days * 100), 1)}%" if total_working_days > 0
else "0.0%"

summary.append({
'id': uid,
'user_code': user_code,
'name': name,
'total_days': total_working_days,
'present': present_days,
'absent': absent_days,
'percentage': percentage
})

conn.close()
return summary

if __name__ == '__main__':
print("=== STEP 2: db_manager.py SELF-TEST ===")

print("1. Initializing DB...")
init_db()

print("2. Verifying default admin (admin / admin123)...")
if verify_admin("admin", "admin123"):
print(" [OK] Admin verification successful.")
else:
print(" [FAIL] Admin verification failed.")

print("3. Testing user creation...")
uid = add_user("TEST_001", "Test User")
if uid != -1:
print(f" [OK] User created successfully with ID: {uid}.")
else:
print(" [INFO] User already exists.")

print("4. Testing attendance marking...")
success, t = mark_attendance("TEST_001", "Test User")
if success:
print(f" [OK] Attendance marked successfully at {t}.")

else:
print(f" [INFO] Attendance already marked today at {t}.")

print("5. Running stats queries...")
stats = get_daily_attendance_stats()
print(f" [OK] Daily Stats: {stats}")

summary = get_user_attendance_summary()
print(f" [OK] User Summary retrieved. Found {len(summary)} user(s).")

print("=== SELF-TEST COMPLETE ===")
```

Module: app.py

```
import customtkinter as ctk
import cv2
import threading
import time
import csv
from tkinter import messagebox, filedialog
from PIL import Image
import numpy as np

# Import our backend modules
import db_manager as db
from vector_index import VectorIndex
from face_engine import FaceEngine

class BiometricDashboard(ctk.CTk):
def __init__(self):
super().__init__()

# 1. Palette & Modern Dark Aesthetic
self.title("AURA-FACE | Biometric Kiosk & Management")
self.geometry("1240x780")
self.minsize(1240, 780)

ctk.set_appearance_mode("Dark")
# Define custom colors
self.colors = {
"bg_deep": "#0B0F19",
"surface": "#151D2E",
"border": "#22304A",
"green": "#10B981",
"blue": "#3B82F6",
"red": "#EF4444",
"purple": "#8B5CF6",
"text_main": "#F8FAFC",
"text_muted": "#94A3B8"
}
self.configure(fg_color=self.colors["bg_deep"])

# Application State
self.mode = "KIOSK"
self.cap = None
self.camera_running = False
self.enrollment_mode = False
self.enrollment_crops = []
self.enrollment_target = 5
self.enrollment_user_id = None

# Initialize Backend Engines
try:
self.face_engine = FaceEngine()
self.vector_index = VectorIndex()
except Exception as e:
messagebox.showerror("Engine Initialization Error", f"Failed to load AI models:\n{e}")

# UI Setup
self.setup_header()
self.setup_kiosk_view()
self.update_clock()

self.protocol("WM_DELETE_WINDOW", self.on_closing)

def setup_header(self):
self.header_frame = ctk.CTkFrame(self, fg_color=self.colors["surface"], corner_radius=0, height=60)
self.header_frame.pack(fill="x", side="top")

# Branding
title = ctk.CTkLabel(self.header_frame, text="AURA-FACE | Biometric Kiosk & Management",
font=ctk.CTkFont(size=20, weight="bold"), text_color=self.colors["text_main"])
title.pack(side="left", padx=20, pady=15)

# Controls (Right side)
self.admin_btn = ctk.CTkButton(self.header_frame, text="Admin Portal", fg_color="transparent",
```

```
border_width=2, border_color=self.colors["blue"], text_color=self.colors["blue"],
hover_color="#1E3A8A", command=self.open_admin_auth)
self.admin_btn.pack(side="right", padx=20, pady=15)

self.clock_label = ctk.CTkLabel(self.header_frame, text="00:00:00",
font=ctk.CTkFont(family="Consolas", size=18), text_color=self.colors["text_muted"])
self.clock_label.pack(side="right", padx=20, pady=15)

def update_clock(self):
current_time = time.strftime("%H:%M:%S")
self.clock_label.configure(text=current_time)
self.after(1000, self.update_clock)

def setup_kiosk_view(self):
self.main_container = ctk.CTkFrame(self, fg_color="transparent")
self.main_container.pack(fill="both", expand=True, padx=20, pady=20)

# --- Left Card (Camera Viewport) ---
self.left_card = ctk.CTkFrame(self.main_container, fg_color=self.colors["surface"], border_width=1,
border_color=self.colors["border"])
self.left_card.pack(side="left", fill="both", expand=True, padx=(0, 10))

cam_header = ctk.CTkFrame(self.left_card, fg_color="transparent")
cam_header.pack(fill="x", padx=20, pady=15)
ctk.CTkLabel(cam_header, text="Live Scanner Terminal", font=ctk.CTkFont(size=18,
weight="bold")).pack(side="left")
self.cam_badge = ctk.CTkLabel(cam_header, text="● STANDBY", text_color=self.colors["text_muted"],
font=ctk.CTkFont(weight="bold"))
self.cam_badge.pack(side="right")
```

```
self.video_label = ctk.CTkLabel(self.left_card, text="Camera is Idle.\nClick 'Start Attendance
Scan' to begin",
font=ctk.CTkFont(size=16), text_color=self.colors["text_muted"])
self.video_label.pack(fill="both", expand=True, padx=20, pady=10)

cam_controls = ctk.CTkFrame(self.left_card, fg_color="transparent")
cam_controls.pack(fill="x", padx=20, pady=20)

self.start_scan_btn = ctk.CTkButton(cam_controls, text="■ Start Attendance Scan",
fg_color=self.colors["green"],
hover_color="#059669", height=40, font=ctk.CTkFont(weight="bold"), command=self.start_kiosk_camera)
self.start_scan_btn.pack(side="left", expand=True, fill="x", padx=(0, 10))

self.stop_scan_btn = ctk.CTkButton(cam_controls, text="■ Stop", fg_color=self.colors["red"],
hover_color="#B91C1C", height=40, width=100, font=ctk.CTkFont(weight="bold"),
command=self.stop_camera)
self.stop_scan_btn.pack(side="right")

# --- Right Card (Dashboard Analytics & Feed) ---
self.right_card = ctk.CTkFrame(self.main_container, fg_color=self.colors["surface"],
border_width=1, border_color=self.colors["border"])
self.right_card.pack(side="right", fill="both", expand=True, padx=(10, 0))

ctk.CTkLabel(self.right_card, text="Daily Analytics", font=ctk.CTkFont(size=18,
weight="bold")).pack(anchor="w", padx=20, pady=(15, 5))

self.kpi_grid = ctk.CTkFrame(self.right_card, fg_color="transparent")
self.kpi_grid.pack(fill="x", padx=20, pady=10)
self.kpi_grid.columnconfigure((0, 1), weight=1)

self.kpi_total = self.create_kpi_card(self.kpi_grid, "Registered Total", "0", self.colors["blue"],
0, 0)
self.kpi_present = self.create_kpi_card(self.kpi_grid, "Present Today", "0", self.colors["green"],
0, 1)
self.kpi_absent = self.create_kpi_card(self.kpi_grid, "Absent Today", "0", self.colors["red"], 1,
0)
self.kpi_rate = self.create_kpi_card(self.kpi_grid, "Attendance Rate %", "0%",
self.colors["purple"], 1, 1)

ctk.CTkLabel(self.right_card, text="Live Punch Stream", font=ctk.CTkFont(size=18,
weight="bold")).pack(anchor="w", padx=20, pady=(20, 5))

self.stream_scroll = ctk.CTkScrollableFrame(self.right_card, fg_color="transparent")
self.stream_scroll.pack(fill="both", expand=True, padx=20, pady=(0, 20))

self.refresh_dashboard()

def create_kpi_card(self, parent, title, value, color, row, col):
card = ctk.CTkFrame(parent, fg_color=self.colors["bg_deep"], corner_radius=8, border_width=1,
border_color=color)
card.grid(row=row, column=col, sticky="nsew", padx=5, pady=5)
ctk.CTkLabel(card, text=title, text_color=self.colors["text_muted"],
font=ctk.CTkFont(size=12)).pack(anchor="w", padx=10, pady=(10, 0))
val_label = ctk.CTkLabel(card, text=value, text_color=color, font=ctk.CTkFont(size=24,
weight="bold"))
val_label.pack(anchor="w", padx=10, pady=(0, 10))
return val_label

def refresh_dashboard(self):
stats = db.get_daily_attendance_stats()
self.kpi_total.configure(text=str(stats['total']))
self.kpi_present.configure(text=str(stats['present']))
self.kpi_absent.configure(text=str(stats['absent']))
self.kpi_rate.configure(text=f"{stats['percentage']}%")

for widget in self.stream_scroll.winfo_children():
widget.destroy()

records = db.get_today_attendance()
for rec in records:
# rec = (user_code, name, date, time)
```

```
card = ctk.CTkFrame(self.stream_scroll, fg_color=self.colors["bg_deep"], corner_radius=6,
border_width=1, border_color=self.colors["border"])
card.pack(fill="x", pady=4)
info_frame = ctk.CTkFrame(card, fg_color="transparent")
```

```
info_frame.pack(side="left", padx=10, pady=10)
ctk.CTkLabel(info_frame, text=rec[1], font=ctk.CTkFont(weight="bold", size=14)).pack(anchor="w")
ctk.CTkLabel(info_frame, text=f"ID: {rec[0]}", font=ctk.CTkFont(size=12),
text_color=self.colors["text_muted"]).pack(anchor="w")

badge = ctk.CTkFrame(card, fg_color=self.colors["green"], corner_radius=4)
badge.pack(side="right", padx=10)
ctk.CTkLabel(badge, text=rec[3], font=ctk.CTkFont(size=12, weight="bold"),
text_color="#000000").pack(padx=8, pady=4)

# --- KIOSK Camera Logic ---
def start_kiosk_camera(self):
if not self.camera_running:
self.cap = cv2.VideoCapture(0)
self.camera_running = True
self.mode = "KIOSK"
self.cam_badge.configure(text="● SCANNING ACTIVE", text_color=self.colors["green"])
self.video_label.configure(text="")
self.update_kiosk_camera()

def update_kiosk_camera(self):
if not self.camera_running or self.mode != "KIOSK":
return

ret, frame = self.cap.read()
if ret:
# Detect faces
try:
faces = self.face_engine.detect_faces(frame)
except Exception:
faces = None

verified_info = None

if faces is not None:
for face in faces:
x, y, w, h = face[:4].astype(int)
# Expand bounding box slightly for the crop
pad_x = int(w * 0.1)
pad_y = int(h * 0.1)
x1 = max(0, x - pad_x)
y1 = max(0, y - pad_y)
x2 = min(frame.shape[1], x + w + pad_x)
y2 = min(frame.shape[0], y + h + pad_y)

face_crop = frame[y1:y2, x1:x2]

if face_crop.shape[0] > 0 and face_crop.shape[1] > 0:
try:
# Extract embedding
embedding = self.face_engine.extract_embedding(face_crop)

# Search in vector index
uid, dist = self.vector_index.search(embedding, threshold=0.55)

if uid is not None:
user_info = db.get_user_by_id(int(uid))
if user_info:
user_code, name = user_info
verified_info = (user_code, name)

# Draw HUD Box
cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2.putText(frame, f"{name} ({user_code})", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0),
2)
else:
# UID in index but not in DB?
cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 0, 255), 2)
cv2.putText(frame, "Unknown Face", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
else:
cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 0, 255), 2)
cv2.putText(frame, "Unknown Face", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
```

```
except Exception as e:
print(f"Embedding extraction error: {e}")

# Convert to RGB and display
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
img = Image.fromarray(frame_rgb)
ctk_img = ctk.CTkImage(light_image=img, dark_image=img, size=(frame.shape[1], frame.shape[0]))
```

```
self.video_label.imgtk = ctk_img
self.video_label.configure(image=ctk_img, text="")

if verified_info:
# Mark attendance
success, timestamp = db.mark_attendance(verified_info[0], verified_info[1])
self.cam_badge.configure(text=f"✔ VERIFIED: {verified_info[1]}", text_color=self.colors["green"])
self.refresh_dashboard()

# Auto-Off Mechanism
self.camera_running = False
self.after(1500, self.stop_camera)
return

self.after(15, self.update_kiosk_camera)

def stop_camera(self):
self.camera_running = False
if self.cap:
self.cap.release()
self.cap = None
self.video_label.configure(image="", text="Camera is Idle.\nClick 'Start Attendance Scan' to
begin")
self.cam_badge.configure(text="● STANDBY", text_color=self.colors["text_muted"])

def on_closing(self):
self.stop_camera()
self.destroy()

# --- Admin Authentication ---
def open_admin_auth(self):
if hasattr(self, 'admin_window') and self.admin_window is not None and
self.admin_window.winfo_exists():
self.admin_window.focus()
return

dialog = ctk.CTkToplevel(self)
dialog.title("Admin Authentication")
dialog.geometry("320x350")
dialog.transient(self)
dialog.grab_set()
dialog.configure(fg_color=self.colors["bg_deep"])

ctk.CTkLabel(dialog, text="■ Admin Portal", font=ctk.CTkFont(size=20,
weight="bold")).pack(pady=(30, 20))

user_entry = ctk.CTkEntry(dialog, placeholder_text="Username", width=200)
user_entry.pack(pady=10)

pass_entry = ctk.CTkEntry(dialog, placeholder_text="Password", show="*", width=200)
pass_entry.pack(pady=10)

def attempt_login():
if db.verify_admin(user_entry.get(), pass_entry.get()):
dialog.destroy()
self.stop_camera()
self.open_admin_dashboard()
else:
messagebox.showerror("Auth Error", "Invalid Credentials", parent=dialog)

ctk.CTkButton(dialog, text="Login", fg_color=self.colors["blue"], hover_color="#1E3A8A",
command=attempt_login, width=200).pack(pady=(20, 10))

# --- Admin Dashboard ---
def open_admin_dashboard(self):
self.admin_window = ctk.CTkToplevel(self)
self.admin_window.title("Admin Dashboard - AURA-FACE")
self.admin_window.geometry("1000x700")
self.admin_window.minsize(1000, 700)
self.admin_window.transient(self)
self.admin_window.grab_set()
```

```
self.admin_window.configure(fg_color=self.colors["bg_deep"])

# Header
header = ctk.CTkFrame(self.admin_window, fg_color=self.colors["surface"], corner_radius=0,
height=60)
header.pack(fill="x", side="top")
ctk.CTkLabel(header, text="Admin Control Panel", font=ctk.CTkFont(size=20,
weight="bold")).pack(side="left", padx=20, pady=15)

ctk.CTkButton(header, text="Return to Kiosk", fg_color="transparent", border_width=1,
border_color=self.colors["text_muted"],
command=self.admin_window.destroy).pack(side="right", padx=20, pady=15)

# Tabview
self.admin_tabs = ctk.CTkTabview(self.admin_window, fg_color=self.colors["bg_deep"])
self.admin_tabs.pack(fill="both", expand=True, padx=20, pady=20)

self.admin_tabs.add("User Management & Enrollment")
self.admin_tabs.add("Attendance Analytics & Export")

self.setup_enrollment_tab()
self.setup_analytics_tab()

def setup_enrollment_tab(self):
tab = self.admin_tabs.tab("User Management & Enrollment")
```

```
# Enrollment Form
form_frame = ctk.CTkFrame(tab, fg_color=self.colors["surface"], corner_radius=8, border_width=1,
border_color=self.colors["border"])
form_frame.pack(fill="x", pady=(0, 20))

ctk.CTkLabel(form_frame, text="User ID / Roll No:").grid(row=0, column=0, padx=10, pady=(15, 5),
sticky="w")
self.reg_id_entry = ctk.CTkEntry(form_frame, width=150)
self.reg_id_entry.grid(row=1, column=0, padx=10, pady=(0, 15))

ctk.CTkLabel(form_frame, text="Full Name:").grid(row=0, column=1, padx=10, pady=(15, 5),
sticky="w")
self.reg_name_entry = ctk.CTkEntry(form_frame, width=200)
self.reg_name_entry.grid(row=1, column=1, padx=10, pady=(0, 15))

ctk.CTkLabel(form_frame, text="Registration Date:").grid(row=0, column=2, padx=10, pady=(15, 5),
sticky="w")
self.reg_date_entry = ctk.CTkEntry(form_frame, width=150)
self.reg_date_entry.insert(0, time.strftime("%Y-%m-%d"))
self.reg_date_entry.grid(row=1, column=2, padx=10, pady=(0, 15))

self.enroll_btn = ctk.CTkButton(form_frame, text="Enroll Face Biometrics",
fg_color=self.colors["purple"], hover_color="#7C3AED", command=self.start_enrollment_process)
self.enroll_btn.grid(row=1, column=3, padx=20, pady=(0, 15))

self.enroll_status = ctk.CTkLabel(form_frame, text="", text_color=self.colors["green"])
self.enroll_status.grid(row=2, column=0, columnspan=4, pady=(0, 10))

# Enrollment Camera View (Hidden by default)
self.enroll_cam_frame = ctk.CTkFrame(tab, fg_color="transparent")
self.enroll_cam_label = ctk.CTkLabel(self.enroll_cam_frame, text="")
self.enroll_cam_label.pack()

# Table View
self.users_textbox = ctk.CTkTextbox(tab, fg_color=self.colors["surface"], border_width=1,
border_color=self.colors["border"], font=ctk.CTkFont(family="Consolas", size=13))
self.users_textbox.pack(fill="both", expand=True, pady=(0, 10))
self.refresh_users_table()

# User Management View
manage_frame = ctk.CTkFrame(tab, fg_color=self.colors["surface"], corner_radius=8, border_width=1,
border_color=self.colors["border"])
manage_frame.pack(fill="x", pady=(0, 10))

ctk.CTkLabel(manage_frame, text="Manage User (Enter UID):").grid(row=0, column=0, padx=10, pady=10,
sticky="w")
self.manage_uid_entry = ctk.CTkEntry(manage_frame, width=80)
self.manage_uid_entry.grid(row=0, column=1, padx=10, pady=10)

ctk.CTkButton(manage_frame, text="Edit Details", fg_color=self.colors["blue"],
command=self.open_edit_user_dialog).grid(row=0, column=2, padx=10, pady=10)
ctk.CTkButton(manage_frame, text="Delete User", fg_color=self.colors["red"],
command=self.delete_user_action).grid(row=0, column=3, padx=10, pady=10)

def refresh_users_table(self):
if not hasattr(self, 'users_textbox'): return
self.users_textbox.configure(state="normal")
self.users_textbox.delete("1.0", "end")
self.users_textbox.insert("end", f"{'S.No':<5} | {'UID':<6} | {'User Code':<15} | {'Full Name':<25}
| {'Reg Date'}\n")
self.users_textbox.insert("end", "-"*80 + "\n")
users = db.get_all_users()
for i, u in enumerate(users, start=1):
self.users_textbox.insert("end", f"{i:<5} | {u[0]:<6} | {u[1]:<15} | {u[2]:<25} | {u[3]}\n")
self.users_textbox.configure(state="disabled")

def open_edit_user_dialog(self):
uid_str = self.manage_uid_entry.get().strip()
if not uid_str.isdigit():
messagebox.showerror("Error", "Please enter a valid numeric UID")
return
```

```
uid = int(uid_str)
user = db.get_user_by_id(uid)
if not user:
messagebox.showerror("Error", "User not found")
return

dialog = ctk.CTkToplevel(self.admin_window)
```

```
dialog.title("Edit User")
dialog.geometry("350x300")
dialog.transient(self.admin_window)
dialog.grab_set()

ctk.CTkLabel(dialog, text=f"Editing User UID: {uid}", font=ctk.CTkFont(size=16,
weight="bold")).pack(pady=20)

code_entry = ctk.CTkEntry(dialog, placeholder_text="User Code", width=200)
code_entry.insert(0, user[0])
code_entry.pack(pady=10)

name_entry = ctk.CTkEntry(dialog, placeholder_text="Full Name", width=200)
name_entry.insert(0, user[1])
name_entry.pack(pady=10)

# Fetch reg_date from DB
user_full = next((u for u in db.get_all_users() if u[0] == uid), None)
reg_date = user_full[3] if user_full else ""

date_entry = ctk.CTkEntry(dialog, placeholder_text="Registration Date", width=200)
date_entry.insert(0, reg_date)
date_entry.pack(pady=10)

def save_changes():
success, msg = db.update_user(uid, code_entry.get().strip(), name_entry.get().strip(),
date_entry.get().strip())
if success:
messagebox.showinfo("Success", "User updated successfully", parent=dialog)
dialog.destroy()
self.refresh_users_table()
self.manage_uid_entry.delete(0, 'end')
else:
messagebox.showerror("Error", msg, parent=dialog)

ctk.CTkButton(dialog, text="Save Changes", command=save_changes,
fg_color=self.colors["green"]).pack(pady=20)

def delete_user_action(self):
uid_str = self.manage_uid_entry.get().strip()
if not uid_str.isdigit():
messagebox.showerror("Error", "Please enter a valid numeric UID")
return

uid = int(uid_str)
user = db.get_user_by_id(uid)
if not user:
messagebox.showerror("Error", "User not found")
return

if messagebox.askyesno("Confirm Delete", f"Are you sure you want to delete user {user[1]}
({user[0]})?\nThis will also remove their attendance and biometric data."):
success, msg = db.delete_user(uid)
if success:
self.vector_index.remove_vector(uid)
messagebox.showinfo("Success", "User deleted successfully")
self.refresh_users_table()
self.manage_uid_entry.delete(0, 'end')
else:
messagebox.showerror("Error", msg)


def start_enrollment_process(self):
code = self.reg_id_entry.get().strip()
name = self.reg_name_entry.get().strip()
reg_date = self.reg_date_entry.get().strip()

if not code or not name:
messagebox.showerror("Error", "Please fill in User ID and Name")
return

# Register in DB
```

```
numeric_id = db.add_user(code, name, reg_date)
if numeric_id == -1:
messagebox.showerror("Error", "User Code already exists!")
return

self.enrollment_user_id = numeric_id
self.enrollment_crops = []

self.enroll_status.configure(text="Please look at the camera. Capturing 5 face samples...",
text_color=self.colors["blue"])
self.enroll_btn.configure(state="disabled")

self.enroll_cam_frame.pack(fill="x", pady=10)
self.users_textbox.pack_forget()

self.mode = "ENROLL"
self.cap = cv2.VideoCapture(0)
self.camera_running = True
self.update_enrollment_camera()

def update_enrollment_camera(self):
if not self.camera_running or self.mode != "ENROLL":
return

ret, frame = self.cap.read()
if ret:
try:
faces = self.face_engine.detect_faces(frame)
except Exception:
faces = None

if faces is not None and len(faces) > 0:
```

```
# Use the largest face
face = max(faces, key=lambda f: f[2]*f[3])
x, y, w, h = face[:4].astype(int)

pad_x = int(w * 0.1)
pad_y = int(h * 0.1)
x1 = max(0, x - pad_x)
y1 = max(0, y - pad_y)
x2 = min(frame.shape[1], x + w + pad_x)
y2 = min(frame.shape[0], y + h + pad_y)

face_crop = frame[y1:y2, x1:x2]
if face_crop.shape[0] > 0 and face_crop.shape[1] > 0:
embedding = self.face_engine.extract_embedding(face_crop)
self.enrollment_crops.append(embedding)

cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2.putText(frame, f"Sample {len(self.enrollment_crops)}/{self.enrollment_target}", (x, y-10),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

# Show preview
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
img = Image.fromarray(frame_rgb)
ctk_img = ctk.CTkImage(light_image=img, dark_image=img, size=(640, 480))
self.enroll_cam_label.configure(image=ctk_img)

if len(self.enrollment_crops) >= self.enrollment_target:
self.finish_enrollment()
return

# 300ms delay to capture different expressions
self.after(300, self.update_enrollment_camera)

def finish_enrollment(self):
self.camera_running = False
if self.cap:
self.cap.release()
self.cap = None

self.enroll_cam_frame.pack_forget()
self.users_textbox.pack(fill="both", expand=True)
self.enroll_btn.configure(state="normal")

# Calculate mean embedding
mean_emb = np.mean(self.enrollment_crops, axis=0).astype(np.float32)
# Re-normalize
norm = np.linalg.norm(mean_emb)
if norm > 0:
mean_emb = mean_emb / norm

# Check for duplicate face before adding
try:
uid, dist = self.vector_index.search(mean_emb, threshold=0.15)
if dist is not None:
print(f"[Enrollment Check] Closest existing vector distance: {dist:.4f} (Threshold: 0.15)")

if uid is not None:
# Duplicate found! Rollback the DB entry
db.delete_user(self.enrollment_user_id)
self.enroll_status.configure(text=f"Error: Face already registered to ID {uid}",
text_color=self.colors["red"])
self.refresh_users_table()
return
except Exception as e:
print(f"Error checking for duplicates: {e}")

# Add to FAISS Index
try:
self.vector_index.add_vector(self.enrollment_user_id, mean_emb)
self.enroll_status.configure(text="✔ User Biometrics Enrolled Successfully!",
text_color=self.colors["green"])
self.reg_id_entry.delete(0, 'end')
```

```
self.reg_name_entry.delete(0, 'end')
except Exception as e:
self.enroll_status.configure(text=f"Error saving to index: {e}", text_color=self.colors["red"])

self.refresh_users_table()

def setup_analytics_tab(self):
tab = self.admin_tabs.tab("Attendance Analytics & Export")

ctrl_frame = ctk.CTkFrame(tab, fg_color="transparent")
ctrl_frame.pack(fill="x", pady=(0, 10))

ctk.CTkButton(ctrl_frame, text="Export Today's Sheet (CSV)", command=self.export_today_csv,
fg_color=self.colors["blue"]).pack(side="left", padx=5)
ctk.CTkButton(ctrl_frame, text="Export Summary (CSV)", command=self.export_summary_csv,
fg_color=self.colors["purple"]).pack(side="left", padx=5)

self.analytics_textbox = ctk.CTkTextbox(tab, fg_color=self.colors["surface"], border_width=1,
border_color=self.colors["border"], font=ctk.CTkFont(family="Consolas", size=13))
```

```
self.analytics_textbox.pack(fill="both", expand=True)

# Load Analytics Data
self.refresh_analytics_table()

def refresh_analytics_table(self):
if not hasattr(self, 'analytics_textbox'): return
summary = db.get_user_attendance_summary()
self.analytics_textbox.configure(state="normal")
self.analytics_textbox.delete("1.0", "end")
self.analytics_textbox.insert("end", f"{'User Code':<15} | {'Name':<25} | {'Total Days':<12} |
{'Present':<10} | {'Absent':<10} | {'Attendance %'}\n")
self.analytics_textbox.insert("end", "-"*95 + "\n")
for s in summary:
self.analytics_textbox.insert("end", f"{s['user_code']:<15} | {s['name']:<25} |
{s['total_days']:<12} | {s['present']:<10} | {s['absent']:<10} | {s['percentage']}%\n")
self.analytics_textbox.configure(state="disabled")

def export_today_csv(self):
records = db.get_today_attendance()
if not records:
messagebox.showinfo("Info", "No attendance records today.")
return

today = time.strftime("%Y-%m-%d")
path = filedialog.asksaveasfilename(defaultextension=".csv", initialfile=f"Attendance_{today}.csv")
if path:
with open(path, 'w', newline='') as f:
w = csv.writer(f)
w.writerow(["User Code", "Name", "Date", "Time"])
w.writerows(records)
messagebox.showinfo("Success", f"Exported to {path}")

def export_summary_csv(self):
summary = db.get_user_attendance_summary()
if not summary:
messagebox.showinfo("Info", "No data to export.")
return

today = time.strftime("%Y-%m-%d")
path = filedialog.asksaveasfilename(defaultextension=".csv",
initialfile=f"Attendance_Summary_{today}.csv")
if path:
with open(path, 'w', newline='') as f:
w = csv.writer(f)
w.writerow(["User Code", "Name", "Total Days", "Present", "Absent", "Attendance %"])
for s in summary:
w.writerow([s['user_code'], s['name'], s['total_days'], s['present'], s['absent'],
s['percentage']])
messagebox.showinfo("Success", f"Exported to {path}")

if __name__ == "__main__":
app = BiometricDashboard()
app.mainloop()
```

Module: evaluate_accuracy.py

```
import os
import cv2
import numpy as np
import db_manager as db
from vector_index import VectorIndex
from face_engine import FaceEngine

def evaluate_pipeline(eval_dir="evaluation_data", threshold=0.55):
db.init_db()
v_index = VectorIndex()
engine = FaceEngine()

tp = 0 # True Positive: Registered user sahi identify hua
fp = 0 # False Positive: Kisi aur ka face galat match ho gaya
fn = 0 # False Negative: Registered user ko Unknown bol diya
tn = 0 # True Negative: Unknown person ko correctly Unknown reject kiya

# 1. Test Registered Faces
reg_path = os.path.join(eval_dir, "registered")
if os.path.exists(reg_path):
for user_code in os.listdir(reg_path):
person_folder = os.path.join(reg_path, user_code)
if not os.path.isdir(person_folder):
continue

for img_name in os.listdir(person_folder):
img_path = os.path.join(person_folder, img_name)
img = cv2.imread(img_path)
if img is None:
continue

faces = engine.detect_faces(img)
if faces is None or len(faces) == 0:
fn += 1
continue

# Crop & extract embedding
x, y, w, h = int(faces[0][0]), int(faces[0][1]), int(faces[0][2]), int(faces[0][3])
crop = img[max(0, y):max(0, y)+h, max(0, x):max(0, x)+w]
if crop.size == 0:
fn += 1
continue

emb = engine.extract_embedding(crop)
matched_id, dist = v_index.search(emb, threshold=threshold)

if matched_id is not None:
pred_code, _ = db.get_user_by_id(matched_id)
if pred_code == user_code:
tp += 1
else:
fp += 1 # Galat user match ho gaya
else:
fn += 1 # Genuine user reject ho gaya

# 2. Test Unknown Faces
unk_path = os.path.join(eval_dir, "unknown")
if os.path.exists(unk_path):
for img_name in os.listdir(unk_path):
img_path = os.path.join(unk_path, img_name)
img = cv2.imread(img_path)
if img is None:
continue

faces = engine.detect_faces(img)
if faces is None or len(faces) == 0:
tn += 1
continue

x, y, w, h = int(faces[0][0]), int(faces[0][1]), int(faces[0][2]), int(faces[0][3])
crop = img[max(0, y):max(0, y)+h, max(0, x):max(0, x)+w]
if crop.size == 0:
```

```
tn += 1
continue

emb = engine.extract_embedding(crop)
matched_id, dist = v_index.search(emb, threshold=threshold)

if matched_id is not None:
fp += 1 # Unknown person ko registered maan liya
else:
tn += 1 # Correctly rejected

total_samples = tp + fp + fn + tn
if total_samples == 0:
print("Koi test images nahi mili. evaluation_data folder me images add karein.")
return

accuracy = ((tp + tn) / total_samples) * 100
precision = (tp / (tp + fp) * 100) if (tp + fp) > 0 else 0.0
recall = (tp / (tp + fn) * 100) if (tp + fn) > 0 else 0.0
far = (fp / (fp + tn) * 100) if (fp + tn) > 0 else 0.0
frr = (fn / (tp + fn) * 100) if (tp + fn) > 0 else 0.0

print("=" * 45)
print(" FACE RECOGNITION EVALUATION REPORT ")
print("=" * 45)
print(f"Total Test Images: {total_samples}")
print(f"True Positives (Correct Matches): {tp}")
print(f"True Negatives (Correct Unknowns): {tn}")
print(f"False Positives (Wrong Identity): {fp}")
print(f"False Negatives (Missed Genuine): {fn}")
print("-" * 45)
print(f"Overall Accuracy : {accuracy:.2f}%")
print(f"Precision : {precision:.2f}%")
print(f"Recall (TPR) : {recall:.2f}%")

print(f"FAR (False Accept): {far:.2f}%")
print(f"FRR (False Reject): {frr:.2f}%")
print("=" * 45)

if __name__ == "__main__":
evaluate_pipeline()
```

Module: resequence_uids.py

```
import sqlite3
import numpy as np
import faiss
from vector_index import VectorIndex

def resequence_all_uids():
conn = sqlite3.connect("attendance.db")
cursor = conn.cursor()

# 1. Fetch existing sorted users
cursor.execute("SELECT id, user_code, name, registered_date FROM users ORDER BY id ASC")
users = cursor.fetchall()

if not users:
print("No users found.")
conn.close()
return

# Map old ID to new ID
id_map = {}
new_users_data = []
for new_id, user in enumerate(users, start=1):
old_id, code, name, reg_date = user
id_map[old_id] = new_id
new_users_data.append((new_id, code, name, reg_date))

# 2. Re-index vectors in FAISS
v_index = VectorIndex()
total_vectors = v_index.index.ntotal

new_index = faiss.IndexIDMap2(faiss.IndexFlatL2(128))

for old_id, new_id in id_map.items():
try:
# Reconstruct old vector using its internal FAISS ID
vec = v_index.index.reconstruct(int(old_id)).reshape(1, -1)
new_index.add_with_ids(vec, np.array([new_id], dtype=np.int64))
except Exception as e:
print(f"Warning: Failed to recover vector for Old UID {old_id}: {e}")

# Save new FAISS index
faiss.write_index(new_index, "models/face_index.faiss")

# 3. Clean and re-insert into SQLite
cursor.execute("DROP TABLE IF EXISTS users_backup")
cursor.execute("CREATE TABLE users_temp (id INTEGER PRIMARY KEY, user_code TEXT UNIQUE, name TEXT,
registered_date TEXT)")
cursor.executemany("INSERT INTO users_temp (id, user_code, name, registered_date) VALUES (?, ?, ?,
?)", new_users_data)

cursor.execute("DROP TABLE users")
cursor.execute("ALTER TABLE users_temp RENAME TO users")

# Reset SQLite autoincrement sequence
cursor.execute("DELETE FROM sqlite_sequence WHERE name='users'")
cursor.execute("INSERT INTO sqlite_sequence (name, seq) VALUES ('users', ?)",
(len(new_users_data),))

conn.commit()
conn.close()

print(f"Success! {len(new_users_data)} users re-sequenced to UIDs 1 to {len(new_users_data)} in
both SQLite and FAISS.")

if __name__ == "__main__":
resequence_all_uids()
```

4. Database Schemas & FAISS Synchronization

Users Table: id (PK), user_code (UNIQUE), name, registered_date. Attendance Table tracks unique punches per day.

ID Resequencing logic maps fragmented SQLite IDs sequentially and reconstructs the FAISS index to ensure harmony.

5. Testing, Verification, and Troubleshooting Matrix

Symptom	Root Cause	Resolution
High False Rejection Rate (FRR)	Kiosk threshold too low (e.g. 0.20)	Revert Attendance threshold to 0.55.
Duplicate Prevention Blocking Friends	Duplicate check threshold too high	Lower duplicate threshold to 0.15.
OpenCV DNN Warnings spamming console	YuNet ONNX backend verbosity	Use cv2.utils.logging.setLogLevel(cv2.utils.logging.LOG_LEVEL_ERROR).